

1. Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

To create multiple child processes and synchronize their completion using wait() and waitpid(), and compare their behavior.

Program using wait()

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
int main() {
    pid_t pid;
    for (int i = 1; i <= 3; i++) {
        pid = fork();
        if (pid == 0) {
            printf("Child %d: PID = %d\n", i, getpid());
            sleep(i);
            printf("Child %d completed\n", i);
            exit(0);
        }
    }
    for (int i = 0; i < 3; i++) {
        pid = wait(NULL);
        printf("Parent collected child PID = %d\n", pid);
    }
    printf("All children completed.\n");
    return 0;}
```

To create and observe a zombie process and eliminate it using proper synchronization.

Output :

pranav@Pranavs-MacBook-Air dir1 % ./zombie1

Child 1: PID = 36808

Child 2: PID = 36809

Child 3: PID = 36810

Child 1 completed

Parent collected child PID = 36808

Child 2 completed

Parent collected child PID = 36809

Child 3 completed

Parent collected child PID = 36810

All children completed.

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
int main() {
    pid_t pid = fork();
    if (pid == 0) {
        printf("Child PID = %d\n", getpid());
        printf("Child exiting...\n");
        exit(0);
    }
    else {
        printf("Parent PID = %d\n", getpid());
        printf("Parent sleeping...\n");
        sleep(20);
        printf("Parent completed.\n");
    }
    return 0;
}
```

output:

pranav@Pranavs-MacBook-Air dir1 % ./modzombie1

Parent PID = 36847

Parent sleeping...

Child PID = 36848

Child exiting...

Parent completed.